

# RecallOps



The Incident Response Agent that learns from every outage

Complete build specification and winning checklist

HackwithHyderabad 3.0 | AI Agents That Learn Using Hindsight

Contents: capabilities, memory design, architecture, data,

UI, demo script, judging map, edge cases, submission checklist, timeline

## Core promise:

Incident 1: generic advice. Incident 5: cites the past fix. Incident 10: warns what failed.

# 1. The Pitch

When production breaks at 3 AM, the on-call engineer re-discovers fixes that someone on the team already found months ago. The knowledge lives in scattered postmortems, Slack threads and people's heads.

**RecallOps** is an incident response agent built on Hindsight memory. It remembers every past incident, its root cause, the fixes that were tried, and whether each fix actually worked. When a new alert fires, it recalls similar incidents, ranks fixes by their real track record, and warns about approaches that failed before. Every resolved incident makes the next one faster.

## One-line pitch for judges

*"PagerDuty tells you something is broken. RecallOps tells you how your team fixed it last time, and what not to try again."*

## Target user and business case

- **Persona:** on-call SRE / backend engineer at a 20-200 person SaaS company.
- **Pain:** slow Mean Time To Resolution (MTTR), repeated incidents, knowledge lost when engineers leave.
- **Pay test:** teams already pay for PagerDuty, Opsgenie, incident.io. \$50/month per team to cut MTTR is an easy yes.
- **Metric to show:** simulated MTTR drops across the demo (e.g. 47 min to 9 min).

# 2. What the Agent Must Be Able to Do (Core - must ship)

| #  | Capability                   | What it does                                                                                                                                    | Hindsight role   |
|----|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| 1  | Alert intake                 | Accepts an alert (service, severity, error message, log snippet, metrics) via UI form or POST /alert. Normalises it into a structured incident. | -                |
| 2  | Similar-incident recall      | Finds past incidents with matching service, error signature, or symptoms and shows them with dates and IDs (e.g. INC-142, 3 weeks ago).         | recall           |
| 3  | Triage briefing              | Generates a short briefing: likely root cause, blast radius, first 3 actions, who fixed it last time.                                           | recall + LLM     |
| 4  | Outcome-ranked fixes         | Lists candidate fixes ranked by past success rate (worked / partial / failed counts), not just similarity.                                      | recall + scoring |
| 5  | Failure warnings             | Explicitly warns: "Rollback failed 2 of 2 times for payments-api; try X first."                                                                 | recall           |
| 6  | Resolution feedback          | Engineer marks each suggested fix as Worked / Partial / Failed and adds notes. This is the learning signal.                                     | retain           |
| 7  | Auto postmortem              | On resolve, drafts a postmortem (timeline, root cause, fix, action items) and stores it in memory.                                              | retain           |
| 8  | Conversational follow-up     | Engineer can ask: "Has this happened after a deploy before?" and get memory-grounded answers.                                                   | recall           |
| 9  | Memory evidence panel        | Every answer shows exactly which memories were used, with source incident IDs. No black box.                                                    | recall output    |
| 10 | With / without memory toggle | Same alert answered twice side by side: stateless LLM vs RecallOps. The single most important demo moment.                                      | on / off         |

### 3. Winning Capabilities (the "extraordinary" layer)

These separate RecallOps from teams doing plain retrieval. Build them in priority order; ship at least A, B and C.

| Pri | Capability                                                                                                                                  | Why judges care                                                     |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| A   | <b>Learning curve dashboard</b> - chart of MTTR and suggestion accuracy across incidents 1..N.                                              | Directly proves "agent improves over time" (25% memory criterion).  |
| B   | <b>Pattern insights via reflect</b> - "payments-api fails within 2h of Friday deploys 4 times"; "DB pool exhaustion recurs every ~3 weeks". | Goes beyond recall into learned knowledge; strong Innovation score. |
| C   | <b>Team knowledge</b> - "Priya resolved 3 similar Redis incidents; loop her in."                                                            | Shows memory of people, not just data; very real-world.             |
| D   | <b>Deploy-risk check</b> - before a deploy, paste the change summary; agent warns about past incidents tied to similar changes.             | Moves from reactive to proactive; great closing demo beat.          |
| E   | <b>Runbook evolution</b> - agent proposes edits to a runbook when the documented step keeps failing in practice.                            | Memory changing documentation = visible learning.                   |
| F   | <b>Recurring-incident detector</b> - flags "this is the 4th time; the permanent fix from INC-098 was never done."                           | Business value: prevents repeats, not just speeds fixes.            |

### 4. Hindsight Memory Design

Memory must be the star. Design it deliberately and explain it in the README and demo.

#### Memory banks

| Bank             | Contents                                                                                         | Why separate                     |
|------------------|--------------------------------------------------------------------------------------------------|----------------------------------|
| incidents        | Every alert, symptoms, root cause, final resolution, postmortem text.                            | Core recall for similarity.      |
| fix-outcomes     | Each fix attempt: service, fix type, outcome (worked/partial/failed), time-to-effect, notes.     | Enables ranking by track record. |
| team             | Who resolved what, expertise areas, preferences (e.g. 'prefers feature-flag off over rollback'). | People-aware suggestions.        |
| baseline (empty) | Used for the no-memory comparison.                                                               | Clean before/after demo.         |

#### When each Hindsight operation is used

- **retain**: on seed load, on every fix feedback click, on resolve (postmortem), on any engineer note. Include metadata: service, severity, date, incident ID, outcome.
- **recall**: on every new alert (query built from service + error signature + symptoms), and on every chat question.
- **reflect**: for the Pattern Insights panel and the weekly-style summary; asks the memory to reason across many incidents.
- **Temporal context**: store real dates so the agent can say "3 weeks ago" and detect recurrence intervals.

#### What 'learning' concretely means (say this to judges)

| Stage                     | Agent behaviour                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| Incident 1 (empty memory) | Generic checklist: check logs, check recent deploys, restart service.                                           |
| Incident 5                | "Matches INC-142 (DB connection pool exhaustion after v2.3 deploy). Increasing pool size resolved it in 8 min." |
| Incident 10               | "Rollback failed twice for this pattern. Pool resize worked 3/3. Priya fixed the last two - page her."          |
| Incident 20               | "This is recurring every ~3 weeks. Permanent fix (connection leak in order-worker) is still open from INC-098." |

# 5. Architecture and Tech Stack

| Layer    | Choice                                             | Notes                                                                              |
|----------|----------------------------------------------------|------------------------------------------------------------------------------------|
| Memory   | Hindsight Cloud (promo MEMHACK99, \$50 credits)    | Fastest setup. Open-source version as fallback.                                    |
| LLM      | Groq - openai/gpt-oss-120b (backup qwen/qwen3-32b) | Wrap tool/function calls with retry + JSON repair + fallback model.                |
| Backend  | Python FastAPI                                     | Endpoints: /alert, /chat, /feedback, /resolve, /insights, /metrics, /seed, /reset. |
| Frontend | React + Tailwind (or single HTML)                  | Dark ops-dashboard look. Charts via Recharts/Chart.js.                             |
| Data     | JSON seed files generated by LLM, hand-checked     | Loaded into Hindsight via /seed script.                                            |
| Deploy   | Render / Railway (backend), Vercel (frontend)      | A live URL impresses more than localhost.                                          |

## Request flow for a new alert

Alert in → normalise → Hindsight recall (incidents + fix-outcomes + team) → score fixes by success rate × similarity → LLM writes briefing using only recalled evidence → UI shows briefing + evidence panel → engineer feedback → Hindsight retain → metrics update.

## Fix ranking formula (simple, explainable)

score = similarity × (worked + 0.5×partial + 1) / (attempts + 2) – 0.3 × recent\_failures. Show the numbers in the UI so the ranking is transparent.

# 6. Realistic Data (the #1 thing that makes it look real)

- **Company:** a fictional SaaS, e.g. "ShipFast" (food delivery platform) with 5 services: payments-api, order-worker, auth-service, search-api, notification-service.
- **Infra:** Postgres, Redis, Kafka, Kubernetes, AWS - use real error formats.
- **20-25 seed incidents** spanning ~4 months, with INC IDs, timestamps, severity (SEV1-SEV3), realistic logs and stack traces.
- **Recurring patterns planted on purpose** (so reflect finds them): DB pool exhaustion after deploys; Redis OOM on Mondays; Kafka consumer lag during sales; expired TLS cert; bad config flag.
- **Mixed outcomes:** some fixes worked, some failed, some partial - otherwise ranking looks fake.
- **6-8 engineers** with Indian and global names and specialties.
- **5 live demo alerts** held back from seed, designed to trigger each capability.

## Example log snippet style

```
2026-09-12T02:14:07Z ERROR payments-api [pool-2] org.postgresql.util.PSQLException: FATAL:
remaining connection slots are reserved for non-replication superuser connections
p99_latency=4820ms error_rate=23.4% deploy=v2.3.1 (41 min ago)
```

# 7. UI Screens

| Screen                | Must contain                                                                                                             |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------|
| Live Incident         | Alert card, agent briefing, ranked fixes with success counts, failure warnings, Worked/Partial/Failed buttons, chat box. |
| Memory Evidence panel | Recalled memories with incident IDs, dates and relevance; visible on every answer.                                       |
| Compare mode          | Split view: "Without memory" vs "RecallOps" for the same alert.                                                          |
| Learning dashboard    | MTTR trend, suggestion accuracy trend, memories stored count, incidents handled.                                         |

| Screen           | Must contain                                                           |
|------------------|------------------------------------------------------------------------|
| Pattern Insights | reflect-generated patterns, recurring incidents, open permanent fixes. |
| Incident history | Timeline of all incidents with postmortems; clickable.                 |

Design: dark theme, red/amber/green severity colours, monospace for logs, streaming LLM text, loading states. Must look like a real ops tool, not a chatbot.

## 8. Demo Script (2-3 minutes)

| Time | Beat      | What judges see                                                                                                    |
|------|-----------|--------------------------------------------------------------------------------------------------------------------|
| 0:00 | Hook      | "It's 3 AM. payments-api is down. Your team fixed this exact issue 3 weeks ago - but nobody remembers how."        |
| 0:20 | Before    | Compare mode: stateless LLM gives a generic checklist.                                                             |
| 0:40 | After     | RecallOps names INC-142, root cause, the fix that worked, and the engineer who fixed it. Evidence panel lights up. |
| 1:10 | Learning  | Mark rollback as Failed. Fire a similar alert. Ranking changes live and a warning appears.                         |
| 1:40 | Insight   | Pattern Insights: "4th pool exhaustion after Friday deploys; permanent fix still open."                            |
| 2:10 | Proactive | Deploy-risk check warns before a risky deploy.                                                                     |
| 2:30 | Proof     | Learning dashboard: MTTR 47 min to 9 min across incidents. Close with the pitch line.                              |

## 9. Mapping to Judging Criteria

| Criterion         | Weight | How RecallOps scores                                                                                             |
|-------------------|--------|------------------------------------------------------------------------------------------------------------------|
| Innovation        | 30%    | Outcome-weighted learning, failure warnings, reflect-driven patterns, deploy-risk prediction. Not a chatbot.     |
| Use of Hindsight  | 25%    | retain/recall/reflect all used; multiple banks; evidence panel; with/without toggle; learning curve chart.       |
| Technical         | 20%    | Clean FastAPI modules, typed models, retries for tool-call errors, tests for scoring, clear README with diagram. |
| User Experience   | 15%    | Real ops-dashboard UI, one-click feedback, streaming, story-driven demo.                                         |
| Real-world impact | 10%    | MTTR reduction metric, clear buyer (SaaS eng teams), integration path to PagerDuty/Slack.                        |

## 10. Edge Cases to Handle (Technical score)

- Empty memory: agent says it has no history and gives safe generic steps (this is your 'before').
- No similar incident found: says so honestly instead of forcing a weak match.
- Conflicting past outcomes: shows counts and recommends the more recent / more frequent success.
- LLM function-calling error or malformed JSON: retry, repair, then fall back to the backup model.
- Hindsight unavailable or slow: timeout with graceful message; UI does not freeze.
- Duplicate alerts within minutes: de-duplicate into one incident.
- Hallucination guard: briefing may only cite incident IDs that came back from recall.
- Sensitive data: mask secrets/tokens in logs before retaining.

## 11. Submission Checklist

| Item        | Done when                                                                                                                    |
|-------------|------------------------------------------------------------------------------------------------------------------------------|
| GitHub repo | Clean structure, README with problem, architecture diagram, Hindsight usage section, setup steps, screenshots, .env.example. |

| Item                      | Done when                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------|
| Hindsight explanation     | Dedicated README section + slide: banks, what is retained, when recall/reflect run, before/after example. |
| Demo video                | 2-3 min following Section 8; captions; shows the learning curve.                                          |
| Live demo                 | Deployed URL, seed loaded, /reset endpoint, 5 prepared alerts, offline backup video.                      |
| Article (each member)     | Build story: problem, why memory, architecture, results, lessons.                                         |
| Social post (each member) | Short post with GIF of compare mode + repo link, tag Hindsight/Vectorize and the hackathon.               |
| Video (each member)       | Per the official content guide.                                                                           |
| Follow content guide      | Check the official Hackathon Content Guide for required challenges.                                       |

## 12. Build Timeline for Today

| Block    | Task                                                                         |
|----------|------------------------------------------------------------------------------|
| Hour 1   | Hindsight Cloud setup, Groq key, repo skeleton, generate + review seed data. |
| Hour 2   | Memory layer: seed loader, retain/recall wrappers, banks.                    |
| Hour 3   | FastAPI: /alert briefing, fix ranking, /feedback, /resolve postmortem.       |
| Hour 4-5 | UI: live incident screen, evidence panel, compare mode.                      |
| Hour 6   | reflect Pattern Insights + learning dashboard metrics.                       |
| Hour 7   | Edge cases, error handling, deploy.                                          |
| Hour 8   | README, demo video, content posts, rehearsal.                                |

## 13. What NOT to Build (scope guard)

- Real PagerDuty/Datadog integrations - mention as roadmap only.
- Authentication, multi-tenant accounts, billing.
- Auto-executing fixes on real infrastructure - the agent recommends, humans act.
- More than one persona or workflow. One thing, done brilliantly.

Win condition: a judge watches 60 seconds and clearly sees the agent get smarter because of memory.